PART A — QA TESTING & QUALITY ASSURANCE [50 Points]

SECTION 1 — TEST PLANNING & ANALYSIS [15 pts]

Task 1.1 — Test Plan

1.1.1 Scope & Objectives

Objective

The goal is to test the entire e-commerce website to make sure everything works properly and meets all required standards before it goes live.

What is Included (In-Scope)

We will test the following features:

- User sign-up and login and edge cases (including login through social media like Google or Facebook)
- Browsing products, searching, and applying filters , pricing , discounts , size selections and booking out of stock products .
- Shopping cart actions (adding items, removing items, changing quantity)
- Applying and validating promo/discount codes and bank offers
- Checkout process with different payment options (Credit Card, PayPal, Cash on Delivery,emi options)
- Placing orders, receiving confirmation, and tracking orders and return's possibility from different locations
- Admin dashboard features (viewing sales reports, managing orders, managing users , managing products adding removing giving discount and cataloging the things)

What is Not Included (Out-of-Scope)

We will NOT test these areas:

- Internal systems of third-party payment providers (like PayPal back end)
- Server setup, hosting, or Develops-related configurations
- The Content Management System (CMS) used by the marketing team
- Mobile apps (iOS/Android) — only the website's responsive design will be tested
- Legal review for GDPR compliance (this will be handled by the legal team)

1.1.2 Testing Types (6 Required)

1. **Functional Testing**
   Check that all features work correctly as expected, such as login, shopping cart, checkout, and admin functions over all the modules intigating correctly and even in database storing the details correclty sending to correct information for all dependancy servives .
2. **Regression Testing**
   Make sure that new changes or updates do not break any existing features that were already working end to end testing need to done from singing up to order placement and even return . there will be some main test cases which we do for regression testing .

3. **Performance Testing**
   Test how the system performs under load. For example, pages should load in under 3 seconds, support around 500 users at the same time, and API s should respond in under 500 milliseconds even how it will work with multiple calls for third party serveries failure retry mechanism and how it will be handled duplicate calls and also even load testing with one object .
4. **Security Testing**
   Check the system for security risks like SQL injection, cross-site scripting (XSS), CSRF attacks, session hijacking, and protection against brute-force login attempts and leakage gap and token with API and all the two way security for login .
5. **User Acceptance Testing (UAT)**
   Real users test the system to confirm it meets business needs and works properly before the final release within the all the flow will check in SIT and then UAT and last pre-production. Will give demo over all the business people and product managers .
6. **Accessibility Testing**
   Ensure the website is usable for people with disabilities by following WCAG 2.1 AA standards, including screen reader support, good color contrast, and keyboard navigation.
7. **Cross-Browser Testing**
   Make sure the website works correctly on major browsers like Chrome, Firefox, Safari, and Edge (latest two versions).
8. **API Testing**
   Test back end API s to ensure they return correct responses, proper status codes, handle errors well, and work correctly using tools like Postman or Newman even will have swagger to over all testing teams.

Entry Criteria

Testing should start only after a few basic things are ready:

- All the user stories and their acceptance criteria,Test Cases and Test scenarios have been checked and approved by the Product Owner.
- The testing environment is set up properly and has some sample data to work with or else mocked data for initial stage to verify validation testing.
- Any major or critical issues from earlier sprints have already been fixed and all the gap which is implemented and solutions should get close and it should be aligner with product engineer.
- The automation setup and CI pipeline are in place and working based on configuration.
- The test plan has been reviewed and signed off by the QA Lead and Project Manager.
- Proper user story should get create and PRD should share over all teams and demo should be given to the QA team (if needed) for expected flow

Exit Criteria

Testing will be considered complete when:

- All the planned test cases have been run and all the test cases should get pass as per the requirement .
- There are no critical or major issues still open.
- If anything is open should be alligned with Product manager and giving demo over all funcanitly to the whole team with business stake holder approval for UAT working funcanility
- At least 95% of all reported defects have been fixed and closed.
- The regression tests are mostly passing (around 98% or more).
- Performance is acceptable — pages load in under 3 seconds and error rates stay below 0.5% even under heavy usage.

- Final approval (sign-off) is received from the Product Owner or business stakeholder.

Risk Register

Here are some common risks we might face during testing and how we plan to handle them:

R1: Payment API not available in the test environment
This could happen sometimes and would affect testing a lot.
Plan: Use a mock or dummy payment service for testing. If that's also not available, inform the Project Manager at least 5 days before the sprint ends.

R2: Requirements changing during the sprint
This is quite likely and can cause confusion or rework.
Plan: Finalize (freeze) requirements during sprint planning. Any new changes should be recorded and moved to the next sprint.

R3: Not enough test data for special or edge cases
This might happen and can limit proper testing.
Plan: Create scripts to generate test data and review data needs during sprint planning.

R4: Checkout becoming slow when many users are using it
This could impact user experience if not handled early.
Plan: Run performance tests early in the sprint. If response time is too high (more than 2 seconds), inform the Dev Lead by Day 5.

R5: Issues on Safari or iOS browsers
This is quite common and needs attention.
Plan: Set aside dedicated time for browser testing and use tools like Browser Stack to test on real devices.

Test Environment Requirements

For testing, we'll need a basic setup that covers different systems and scenarios.

- The application should be tested on Windows 11, mac OS Venture (13 or above), and Ubuntu 22.04 so we know it works across common operating systems.
- For browsers, we'll use the latest versions of Chrome, Firefox, Safari, and Edge. Testing on the most recent versions should be enough.
- Mobile testing is also important, so we'll check on devices like iPhone 14 (iOS 17) and Samsung Galaxy S23 (Android 13).
- We'll be using standard testing tools such as Selenium and Cypress for automation, Postman for API testing, J Meter for performance, and JIRA and Test Rail for tracking and test management.
- Some sample test data should already be available in the system — around 10 users with different roles, about 50 products, a few promo codes, and test payment details.
- A stable internet connection is needed, but we should also test how the app behaves on slower networks like 3G.
- The database should be a staging environment with data similar to production, but with all sensitive information removed or anonymize.

Task 1.2 — Test Estimation (Three-Point Technique)

Technique Used: Three-Point Estimation (PERT). Formula: E = (O + 4M + P) / 6

Effort Estimation

Here's a rough idea of how much time testing might take for each module. These are practical estimates based on the type of work involved.

User Authentication (Login / Sign up)

- Test design: ~6 hours
- Test execution: ~8–9 hours
- Defect re-testing: ~3 hours
  Reason: Requirements are clear, but we still need to cover negative cases, edge cases, and session handling and to check end to end flow with actual data.

Product Catalog (Search / Filter / Browse)

- Test design: ~8 hours
- Test execution: ~12 hours
- Defect re-testing: ~4 hours
  Reason: There are many combinations for search and filters, plus UI checks across browsers and responses for all results like search, filters and dependency services .

Checkout (Cart / Payment / Order Placement)

- Test design: ~12 hours
- Test execution: ~16 hours
- Defect re-testing: ~5–6 hours
  Reason: This is the most critical part of the app. Multiple payment methods and scenarios need careful and repeated testing and placing the order like single line single qty muilt line mutily qty various edge cases with different different combinations need to test while creating order even with some like some prodcuts and eligible for discount and some or not like this we have multiple scnarioes .

Admin Dashboard

- Test design: ~6 hours
- Test execution: ~10 hours
- Defect re-testing: ~3 hours
  Reason: Mostly involves data validation, reports, and basic CRUD operations, but still needs careful checking and even of order management system over all end to end system should handle and if not able to handle it should give proper understable error for users .

Overall Estimate

In total, testing will take around **95–100 hours**, which is roughly **12 working days for one QA engineer**.

SECTION 2 — TEST CASE DESIGN [15 pts]

Task 2.1 — Functional Test Cases: User Login Feature

TC-001: Valid Login with Correct Credentials
Description: Please verify user is able to login with valid credentials

Go to login page

Enter valid email and password

Click login

> **Expected:** User should be logged in and redirected to dashboard with welcome message
> **Priority:** P1
> **Status:**

TC-002: Login with Wrong Password (1st & 2nd Attempt)
Description: Please verify error message is shown for wrong password attempts

Enter correct email

Enter wrong password

Click login

Repeat once

> **Expected:** "Invalid credentials" message should be displayed, account should not be locked
> **Priority:** P1
> **Status:**

TC-003: Warning on 3rd Failed Attempt
Description: Please verify warning message is shown on 3rd failed login attempt

Enter valid email

Enter wrong password

Click login

> **Expected:** Warning message should appear for last attempt before lock
> **Priority:** P1
> **Status:**

TC-004: Account Lock after 5 Failed Attempts
Description: Please verify account gets locked after multiple failed attempts

Enter valid email

Enter wrong password

Click login

Expected: Account should be locked and proper message displayed, email notification triggered
Priority: P1
Status:

TC-005: Login with Unregistered Email
Description: Please verify system does not allow login with unregistered email

Enter unregistered email

Enter any password

Click login

> **Expected:** Generic error message should be shown
> **Priority:** P1
> **Status:**

TC-006: Blank Email Field
Description: Please verify validation message when email field is empty

Leave email empty

Enter password

Click login

> **Expected:** "Email is required" message should be displayed
> **Priority:** P2
> **Status:**

TC-007: Blank Password Field
Description: Please verify validation message when password field is empty

Enter email

Leave password empty

Click login

Expected: "Password is required" message should be displayed
Priority: P2
Status:

TC-008: Both Fields Blank
Description: Please verify validation messages when both fields are empty

Leave both fields empty

Click login

Expected: Both validation messages should be displayed
Priority: P2
Status:

TC-009: Remember Me Checked
Description: Please verify session is retained when Remember Me is selected

Enter valid credentials

Check Remember Me

Login

Close and reopen browser

Expected: User should remain logged in
Priority: P2
Status:

TC-010: Remember Me Not Checked
Description: Please verify session is not retained when Remember Me is not selected

Enter valid credentials

Do not check Remember Me

Login

Close and reopen browser

Expected: User should be asked to login again
Priority: P2
Status:

TC-011: Password Case Sensitivity (Correct Case)
Description: Please verify login works with correct password case

Enter valid email

Enter correct case password

Click login

Expected: Login should be successful
Priority: P1
Status:

TC-012: Password Case Sensitivity (Wrong Case)
Description: Please verify login fails with incorrect password case

Enter valid email

Enter wrong case password

Click login

Expected: Login should fail with error message
Priority: P1
Status:

TC-013: Session Timeout
Description: Please verify user is redirected after session timeout

Let session expire

Try accessing any page

Expected: User should be redirected to login and after login taken back to same page
Priority: P2
Status:

TC-014: SQL Injection in Email Field
Description: Please verify system handles SQL injection in email field

Enter SQL query in email

Enter any password

Click login

Expected: Login should fail and no system error should be shown
Priority: P1
Status:

TC-015: SQL Injection in Password Field
Description: Please verify system handles SQL injection in password field

Enter valid email

Enter SQL query in password

Click login

Expected: Login should fail safely without any data issue
Priority: P1
Status:

Task 2.2 — BVA & Equivalence Partitioning

Fields: Username (6–20 chars) | Password (8–32 chars) | Age (18–99)

Field: Username

Description: Please verify username field validation for different inputs

- Valid username (8 chars) → `testuser`
  Expected: Should be accepted
- Min length (6 chars) → `abcdef`
  Expected: Should be accepted
- Less than min length (5 chars) → `abcde`
  Expected: Should show error “Username too short”
- Max length (20 chars) → `abcdefghijklmnopqrst`
  Expected: Should be accepted
- More than max length (21 chars) → `abcdefghijklmnopqrstu`
  Expected: Should show error “Username too long”
- Empty value → `""`
  Expected: Should show “Username required”

Field: Password

Description: Please verify password field validation

- Valid password → `SecureP@1`
  Expected: Should be accepted
- Min length (8 chars) → `Pass@123`
  Expected: Should be accepted
- Less than min length (7 chars) → `Pass@12`
  Expected: Should show error “Password too short”
- Max length (32 chars) → `AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA`
  Expected: Should be accepted
- More than max length (33 chars) → `AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA`
  Expected: Should show error “Password too long”
- Empty value → `""`
  Expected: Should show “Password required”

Field: Age

Description: Please verify age field validation

- Valid age → `30`
  Expected: Should be accepted
- Min age (18) → `18`
  Expected: Should be accepted
- Below min age → `17`
  Expected: Should show error “You must be 18 or older”
- Max age (99) → `99`
  Expected: Should be accepted
- Above max age → `100`
  Expected: Should show error “Invalid age”
- Non-numeric → `twenty`
  Expected: Should show error “Age must be a number”
- Negative value → `-5`
  Expected: Should show error “Invalid age”

SECTION 3 — BUG REPORTING & AUTOMATION [20 pts]

Task 3.1 — Formal Bug Reports

BUG-001 — Promo Code SAVE20 Applies 10% Discount Instead of 20%

Steps to Reproduce

1. Add items totalling more 1000 rupee to cart
2. Navigate to checkout
3. Enter promo code 'SAVE20'
4. Click 'Apply'
5. Observe cart total

Actual Result	Order total reduced by 10% (e.g., 1000 cart shows 900 instead of 800).
Expected Result	SAVE20 should reduce order total by exactly 20% when cart total exceeds 1000.

BUG-002 — 'Place Order' Button Disappears on Scroll — iOS 17 / Safari

Bug ID	BUG-002	**Title**	'Place Order' Button Disappears on Scroll — iOS 17 / Safari
Environment	Production | Safari 17 | iPhone 14 | iOS 17.4	**Severity**	S1 — Critical
Priority	P1 — High	**Attachment**	[Screen recording: ios_safari_placeorder_bug.mp4]

Steps to Reproduce

6. Open app in Safari on iPhone 14 (iOS 17)
7. Add item to cart and proceed to checkout
8. Fill in payment details
9. Scroll down on the payment page
10. Observe the 'Place Order' button

Actual Result	The 'Place Order' button becomes invisible/hidden after scrolling down on the payment page. Users cannot complete checkout.
Expected Result	The 'Place Order' button must remain visible and accessible at all scroll positions on mobile browsers.

BUG-003 — Browser Back Button Re-submits Order Creating Duplicate

Bug ID	BUG-003	**Title**	Browser Back Button Re-submits Order Creating Duplicate
Environment	Staging | Chrome 124 | Windows 11 | Build 2.4.1	**Severity**	S1 — Critical
Priority	P1 — High	**Attachment**	[Screenshot: duplicate_order_bug.png, Network HAR: order_resubmit.har]

Steps to Reproduce

11. Add item to cart and complete checkout
12. Observe order confirmation page (Order #12345)
13. Press browser Back button
14. Observe response

Actual Result	The order is re-submitted, generating a duplicate order (e.g., Order #12346) with identical items and payment charge.
Expected Result	Browser back navigation after order confirmation should display a cached confirmation page or redirect to order history. No re-submission should occur.

BUG-004 — Admin Dashboard Revenue Total Incorrectly Includes Cancelled Orders

Bug ID	BUG-004	**Title**	Admin Dashboard Revenue Total Incorrectly Includes Cancelled Orders
Environment	Staging | Chrome 124 | Windows 11 | Admin Role | Build 2.4.1	**Severity**	S2 — Major
Priority	P1 — High	**Attachment**	[Screenshot: admin_revenue_bug.png, CSV export: orders_export.csv]

Steps to Reproduce

15. Login as Admin and navigate to Dashboard
16. Note the 'Total Revenue' figure shown
17. Navigate to Orders > filter by 'Cancelled'
18. Sum the cancelled order values manually
19. Compare: Dashboard total vs (All Orders − Cancelled Orders)

Actual Result	The dashboard 'Total Revenue' figure includes revenue from cancelled orders, overstating actual revenue.
Expected Result	Total Revenue should only include orders with status 'Completed' or 'Delivered'. Cancelled, refunded, and pending orders must be excluded.

Task 3.2 — Test Automation Script (Selenium Java — POM)

please refer git url : https://github.com/Megha9481/assignment-submission.git